

Laporan Praktikum Minggu Ke-3 Kontrol Cerdas

Minggu ke-3

Nama : Fatmawati Tia Pratama Putri
NIM : 224308056
Kelas : TKA 7C
Akun Github (Tautan) : <https://github.com/fatmawati25>

1. Judul Percobaan

Image Classification with CNN

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

- Memahami konsep dasar *Deep Learning* dalam sistem kendali.
- Mengimplementasikan *Convolutional Neural Network* (CNN) untuk klasifikasi objek.
- Menggunakan *TensorFlow* dan *Keras* untuk membangun model *Deep Learning*.
- Mengintegrasikan model CNN dengan *Computer Vision* untuk deteksi objek secara *real-time*.
- Menggunakan dataset dari Kaggle untuk pelatihan model.
- Mengembangkan mode *Night Vision* untuk deteksi objek dalam kondisi pencahayaan rendah.

3. Landasan Teori

Deep Learning (DL) merupakan subset dari Machine Learning yang menggunakan jaringan saraf tiruan (Neural Networks) untuk belajar dari data. Beberapa penerapannya dalam sistem kendali:

- Autonomous Vehicles: Mendeteksi dan mengenali objek di jalan.
- Robotics: Navigasi robot menggunakan Computer Vision.
- Quality Control: Mendeteksi cacat produk dalam industri manufaktur.
- Smart Surveillance: Pengenalan wajah dan aktivitas manusia.

- **Night Vision:** Mendeteksi objek dalam kondisi pencahayaan rendah dengan teknik pemrosesan gambar khusus.

Inti dari *deep learning* adalah **Jaringan Saraf Tiruan (JST)** atau *Artificial Neural Network (ANN)*. JST meniru cara kerja otak manusia yang terdiri dari banyak neuron yang saling terhubung. Dalam konteks sistem kendali, JST dapat berfungsi sebagai:

- **Model Identifikasi:** JST dapat belajar memodelkan dinamika sistem yang tidak diketahui atau kompleks (misalnya, bagaimana kecepatan motor berubah berdasarkan input daya).
- **Pengendali (Controller):** JST dapat secara langsung bertindak sebagai pengendali untuk menghasilkan sinyal kendali yang optimal berdasarkan sensor input.

CNN adalah jenis jaringan saraf tiruan yang banyak digunakan dalam pemrosesan gambar. Secara ringkas, CNN memproses gambar dengan mengekstraksi fitur-fitur dari data input, lalu menggunakan fitur-fitur tersebut untuk mengklasifikasikan objek. Struktur utamanya terdiri dari:

- *Convolutional Layers:* Mengekstrak fitur dari gambar. Kumpulkan dataset gambar yang beragam, idealnya dengan objek yang sudah diberi label
- *Pooling Layers:* Mengurangi dimensi gambar untuk mempercepat proses. Ubah ukuran (resize) semua gambar menjadi dimensi yang sama (contoh: 64x64 piksel). Normalisasi data (misalnya, mengubah nilai piksel dari 0-255 menjadi 0-1) membantu mempercepat pelatihan.
- *Fully Connected Layers:* Menghubungkan fitur yang diekstrak ke kelas output. Vektor yang sudah diratakan (flattened) dimasukkan ke dalam satu atau lebih lapisan *fully connected* (dense layers). Lapisan ini bertindak seperti Jaringan Saraf Tiruan (JST) biasa dan menggunakan fitur-fitur yang dipelajari oleh lapisan konvolusi untuk melakukan klasifikasi.
- *Activation Functions:* Seperti ReLU dan Softmax untuk keputusan akhir.

Pentingnya CNN dalam Computer Vision adalah kemampuannya untuk “belajar” fitur-fitur yang relevan dari data pelatihan. Misalnya, jika kita ingin CNN untuk mengenali kucing, kita hanya perlu memberikan banyak gambar

kucing sebagai data pelatihan. CNN akan secara otomatis mempelajari fitur-fitur seperti bentuk tubuh, mata, dan telinga kucing tanpa perlu spesifikasi manual.

Selain itu terdapat juga *Night Vision* dalam sistem deteksi objek. Penggunaan *night vision* dalam sistem deteksi objek memungkinkan teknologi ini berfungsi secara efektif dalam kondisi cahaya rendah atau total kegelapan. Teknologi *night vision* pada dasarnya mengubah kondisi gelap menjadi gambar yang dapat dianalisis oleh algoritma deteksi objek. Kamera *night vision* memiliki infra merah (IR) sebagai salah satu komponennya. IR adalah bagian dari spektrum cahaya yang tidak terlihat oleh mata manusia. Kamera ini memanfaatkan infra merah ini untuk membantu melihat objek di tempat yang gelap. Ketika kamera *night vision* menyala, IR akan mengirimkan pulsa cahaya infra merah yang tidak terlihat kepada objek di depannya. ketika pulsa cahaya infra merah tersebut mencapai objek, sebagian besar cahaya itu akan dipantulkan kembali ke kamera. Kamera ini kemudian akan mengolah cahaya yang dipantulkan tersebut menjadi gambar yang bisa kita lihat. Uniknya, kamera *night vision* menggunakan sensor fotodiode yang dapat mendeteksi dan mengubah cahaya inframerah menjadi sinyal elektronik. Sinyal elektronik ini kemudian diubah menjadi gambar oleh kamera.

Konsep dasar Version control dengan GitHub adalah:

- **Git sebagai Mesinnya:** Git adalah perangkat lunak yang diinstal secara lokal di komputer Anda. Ia bertanggung jawab untuk melacak setiap perubahan, membuat versi (*commit*), dan mengelola riwayat kode.
- **GitHub sebagai Platformnya:** GitHub adalah layanan *online* tempat Anda menyimpan salinan repositori Git Anda. Ini berfungsi sebagai "sumber kebenaran" pusat yang memungkinkan tim berkolaborasi.

4. Analisis dan Diskusi

- **Analisis**

1. Bagaimana performa model dalam mendeteksi objek?

Dalam percobaan kali ini, program dapat mengidentifikasi objek dengan cukup baik diantaranya hutuan, bangunan, jalan raya, gunung dan pantai.

2. Bagaimana perbedaan akurasi jika jumlah dataset ditambah?

Akurasi model dalam mendeteksi objek juga bergantung pada kualitas dataset dan jumlah data latih. Dengan dataset kecil, akurasi bisa rendah karena model mungkin tidak mencakup variasi/macam objek yang lebih banyak. Jika jumlah dataset ditambah dengan lebih banyak objek, terutama dengan variasi pencahayaan yang berbeda, akurasi cenderung meningkat karena model memiliki lebih banyak referensi untuk membedakan objek dengan lebih baik.

3. Bagaimana cara meningkatkan kinerja model klasifikasi?

Untuk meningkatkan kinerja model klasifikasi, beberapa langkah yang bisa dilakukan adalah menggunakan jumlah tetangga optimal pada KNN (bisa diuji dengan cross-validation). Menggunakan dataset warna yang lebih lengkap, tidak hanya warna dasar (Red, Green, Blue), tetapi juga turunan (Light Blue, Dark Red, dll). Digunakan juga *Night Vision* agar model dapat mengidentifikasi objek dipencahayaan rendah atau bahkan di malam hari.

- **Diskusi**

1. Apa kelebihan model Deep Learning kali ini?

Dengan model Deep Learning yang dimana merupakan jaringan syaraf tiruan, model dapat mendeteksi apa saja yang ditangkap kamera seperti pengenalan wajah, aktivitas, tempat bahkan bisa mendeteksi cacat produk dalam industri bahkan dengan kondisi pencahayaan rendah.

2. Bagaimana DL dapat diintegrasikan lebih lanjut dalam sistem kendali?

Deep learning dapat diintegrasikan lebih lanjut ke dalam sistem kendali melalui beberapa pendekatan canggih, utamanya dengan memanfaatkan kemampuannya untuk mempelajari model kompleks dari data dan berinteraksi secara adaptif dengan lingkungan.

Integrasi paling revolusioner terjadi melalui ***Deep Reinforcement Learning (DRL)***. Dalam pendekatan ini, sistem kendali (agen) belajar secara mandiri untuk mengendalikan suatu sistem (lingkungan) dengan tujuan memaksimalkan hadiah (imbalan) yang diterima dari lingkungan

tersebut. Ini sangat berguna untuk mengendalikan sistem yang sangat kompleks dan tidak diketahui dinamikanya, seperti robotika canggih yang melakukan tugas-tugas rumit, mobil otonom yang harus beradaptasi dengan kondisi lalu lintas yang tidak terduga, atau mengoptimalkan penggunaan energi di pusat data.

Model *deep learning* dapat digunakan untuk membangun **model prediktif** dari suatu sistem, yang kemudian digunakan oleh pengendali konvensional untuk membuat keputusan yang lebih cerdas. Pengendali dapat menggunakan prediksi ini untuk mengantisipasi gangguan, mencegah masalah, atau mengoptimalkan respons sistem secara proaktif. Misalnya, dalam sistem kendali industri, model *deep learning* dapat memprediksi kegagalan peralatan sebelum terjadi, memungkinkan tindakan pemeliharaan yang tepat waktu..

3. Apa saja tantangan dalam penerapan DL?

Untuk menerapkan pembelajaran ini secara efektif, diperlukan fasilitas yang mendukung, seperti laboratorium yang memadai, perangkat teknologi modern, dan akses internet yang stabil. Tanpa dukungan ini, pembelajaran yang berbasis eksplorasi dan eksperimen akan sulit dilakukan. Proses pelatihan model yang bisa memakan waktu berjam-jam, sehari-hari, atau bahkan berminggu-minggu yang menunda siklus pengembangan dan pengujian.

Model *deep learning* membutuhkan dataset yang sangat besar dan bervariasi untuk dapat belajar secara efektif. Tanpa data yang cukup, model cenderung tidak akurat atau tidak mampu melakukan generalisasi dengan baik. Data yang buruk—berisi *noise*, tidak lengkap, atau tidak konsisten—dapat menyebabkan model membuat prediksi yang salah. Proses pembersihan dan pelabelan data (*data labeling*) seringkali memakan waktu dan biaya yang sangat besar.

Model *deep learning* rentan terhadap serangan di mana input sedikit dimodifikasi untuk menipu model agar membuat prediksi yang salah, seperti mengubah rambu lalu lintas agar tidak dikenali oleh mobil otonom. Ketika sebuah sistem yang didukung oleh *deep learning*

menyebabkan kerugian atau kecelakaan, sulit untuk menentukan siapa yang bertanggung jawab—penulis algoritma, penyedia data, atau pengguna.

5. Assignment

Pada praktikum kali ini, program yang dibuat agar dapat mengidentifikasi objek seperti gunung, pantai, hutan, jalan dan bangunan. Gambar yang dihasilkan bukan hanya gambar objek secara *real* tapi juga berupa tangkapan infra merah (IR). Agar program dapat mengidentifikasi objek dengan tepat, diperlukan dataset. Dataset yang diperoleh dari Kaggle kemudian dimasukkan dalam program. Sempat terjadi kegagalan saat program dijalankan karena dataset tidak terbaca oleh program. Setelah diperbaiki program berjalan sesuai perintah. Diperlukan waktu kurang lebih 30 menit untuk program mengolah data sehingga program dapat berjalan semestinya.

Menggunakan **OpenCV**, program memuat kembali model yang telah disimpan. Lalu, program mengakses webcam untuk mendapatkan *frame* video secara terus-menerus. Setiap *frame* diubah ukurannya dan diproses agar sesuai dengan input model, yang kemudian digunakan untuk memprediksi kelas gambar. Hasil prediksi (label kelas) ditampilkan secara langsung pada layar video bersamaan dengan fitur "Night Vision" sebagai tambahan visual. Secara keseluruhan, program ini menggabungkan dua pustaka utama, **TensorFlow/Keras** untuk membangun dan melatih model *deep learning* dan **OpenCV** untuk pemrosesan video dan gambar *real-time*.

Bagian ini menyiapkan data gambar agar siap untuk dilatih oleh model:

- `ImageDataGenerator` : Kelas dari Keras ini digunakan untuk **augmentasi data**, yaitu proses membuat variasi dari data pelatihan yang sudah ada. Tujuannya adalah untuk meningkatkan jumlah data dan membuat model lebih robust dan tidak terlalu peka terhadap variasi gambar, seperti rotasi, zoom, atau flip horizontal. `rescale=1./255` adalah langkah **normalisasi** yang mengubah nilai piksel dari rentang `[0, 255]` menjadi `[0, 1]`, yang membuat model lebih mudah belajar.

- `flow_from_directory`: Fungsi ini secara otomatis mengambil gambar dari direktori yang ditentukan, membaginya ke dalam batch, dan menghasilkan label berdasarkan struktur folder. Misalnya, jika Anda memiliki folder untuk "mountain" dan "forest", `flow_from_directory` akan secara otomatis membuat label untuk kedua kategori tersebut. `target_size=(150, 150)` akan mengubah ukuran semua gambar menjadi 150x150 piksel, standar yang diperlukan oleh model.

Bagian ini mendefinisikan, mengkompilasi, dan melatih model jaringan saraf tiruan (CNN) untuk klasifikasi gambar:

- `Sequential`: Ini adalah kelas dasar untuk membuat model Keras di mana Anda menambahkan layer secara berurutan, satu per satu.
- `Conv2D` dan `MaxPooling2D`: Ini adalah layer inti dari **Jaringan Saraf Tiruan Konvolusional (CNN)**. `Conv2D` melakukan ekstraksi fitur dari gambar dengan menerapkan filter konvolusi. `MaxPooling2D` mengurangi dimensi data, yang membantu mengurangi kompleksitas komputasi dan mencegah *overfitting*.
- `Flatten`: Layer ini mengubah output multidimensi dari layer konvolusi dan pooling menjadi vektor satu dimensi, sehingga dapat digunakan sebagai input untuk layer `Dense`.
- `Dense`: Ini adalah layer terhubung penuh, di mana setiap neuron terhubung ke setiap neuron di layer sebelumnya. Ini berfungsi untuk klasifikasi akhir. Layer terakhir dengan `Dense(6, activation='softmax')` memiliki 6 neuron, yang sesuai dengan jumlah kelas, dan menggunakan aktivasi softmax untuk menghasilkan probabilitas prediksi untuk setiap kelas.
- `model.compile`: Langkah ini menyiapkan model untuk pelatihan. `optimizer='adam'` adalah algoritma yang menyesuaikan bobot model selama pelatihan. `loss='categorical_crossentropy'` adalah fungsi kerugian yang umum digunakan untuk masalah klasifikasi multi-kelas.

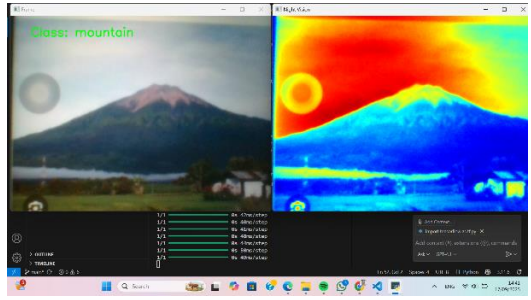
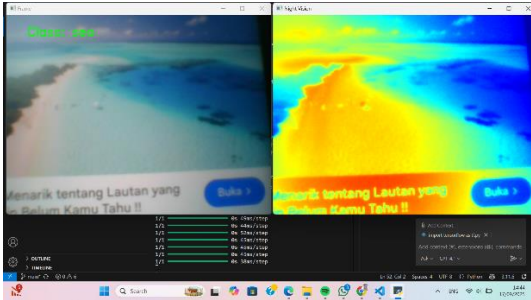
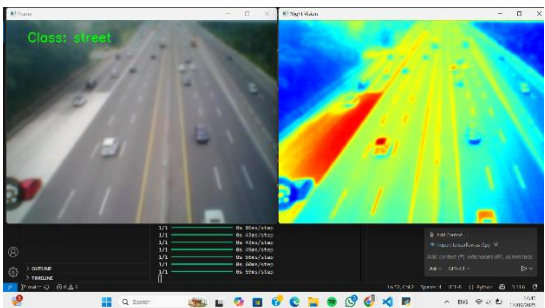
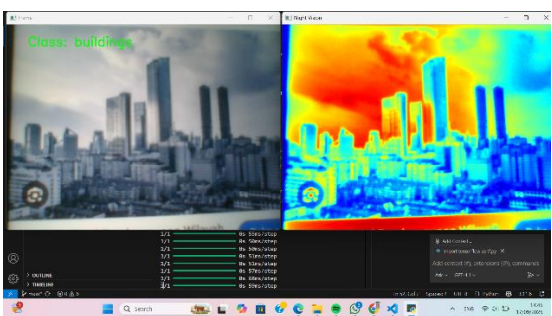
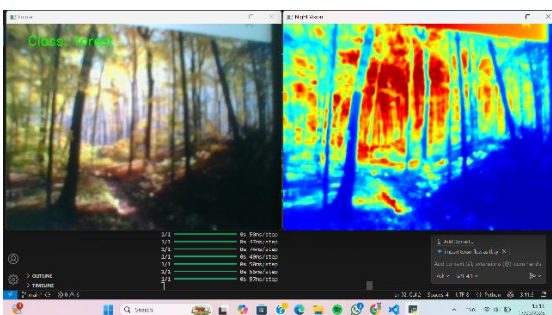
- `model.fit`: Fungsi ini memulai proses pelatihan model. Model akan belajar dari `train_generator` dan dievaluasi performanya menggunakan `val_generator` selama 10 epoch.

Bagian ini menunjukkan bagaimana menggunakan model yang sudah dilatih untuk membuat prediksi pada video *real-time* dari webcam:

- `load_model('cnn_model.h5')`: Model yang telah dilatih dan disimpan di Bagian 2 dimuat kembali untuk digunakan.
- `cv2.VideoCapture(0)`: Kode ini menginisialisasi **kamera (webcam)**. Angka 0 biasanya merujuk pada kamera default.
- `while True`: Ini adalah loop tak terbatas yang akan terus berjalan, mengambil frame dari kamera secara berurutan hingga pengguna mengakhirinya.
- `cv2.resize` dan `np.expand_dims`: Setiap frame yang diambil dari kamera harus diubah ukurannya dan dinormalisasi agar sesuai dengan format yang diharapkan oleh model (150x150 piksel, nilai dalam rentang [0, 1], dan dimensi batch). `np.expand_dims` menambahkan dimensi batch yang diperlukan oleh model.
- `model.predict(img)`: Ini adalah bagian kunci yang menggunakan model yang sudah dilatih untuk memprediksi kelas dari gambar yang diberikan.
- `cv2.putText` dan `cv2.imshow`: Fungsi OpenCV ini digunakan untuk menampilkan hasil prediksi (label kelas) pada layar dan menampilkan frame video ke jendela.
- `cv2.waitKey(1) & 0xFF == ord('q')`: Baris kode ini memungkinkan Anda untuk keluar dari loop dengan menekan tombol 'q' pada keyboard.

6. Data dan Output Hasil Pengamatan

No	Data	Hasil Percobaan
----	------	-----------------

1	Program dapat mengidentifikasi Objek Gunung	
2	Program dapat mengidentifikasi Objek Pantai	
3	Program dapat mengidentifikasi Objek Jalan Raya	
4	Program dapat mengidentifikasi Objek Bangunan	
5	Program dapat mengidentifikasi Objek Hutan	

7. Kesimpulan

Berdasarkan percobaan yang dilakukan dapat disimpulkan bahwa:

- 1) Percobaan ini menggunakan model CNN dengan *Computer Vision* untuk mendeteksi objek secara *Real-Time*.
- 2) Model CNN yang telah dilatih menggunakan dataset yang dapat mendeteksi objek. Tingkat akurasi dihitung berdasarkan perbandingan antara hasil prediksi model terdekat yang terdapat dalam dataset.
- 3) Penggunaan Model *Night Vision* terbukti dapat meningkatkan visibilitas citra pada kondisi pencahayaan rendah
- 4) Pemrosesan data pada konsep *Deep Learning* memerlukan waktu yang tidak sebentar.

8. Saran

Menambah Selain menampilkan label, tampilkan juga seberapa yakin model dengan prediksinya (dalam bentuk persentase). Misalnya, "Mountain (95%)". Ini memberi gambaran seberapa akurat model bekerja. Pasang pengawas agar model berhenti latihan saat sudah mencapai batas maksimalnya. Ini mencegah model jadi terlalu "terlatih" dan justru jadi kurang akurat pada gambar lain. Menggunakan 10 *epoch* mungkin tidak cukup untuk model agar konvergen. Coba tingkatan jumlah *epoch* menjadi 30 atau 50.

9. Daftar Pustaka

- (Leravio, 2025)
<https://leravio.com/blog/deep-learning-untuk-computer-vision/>
- (Dhia, October 8, 2023)
<https://takterlihat.com/kamera-night-vision-cara-kerja/>